Experiment 6

Student Name: Devansh Sharma **UID: 23BAI70166**
Branch: BE CSE-H (AIML) (IBM) **Section/Group: 23AIT_KRG-G- 1**
Semester: 6th **Date of Performance: 27/03/26**
Subject Name: Full Stack - II **Subject Code: 23CSH - 382**

1. Aim:

To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

2. Objective:

1. Understand Spring Boot fundamentals

- o Learn the role of Spring Boot in backend development.
- o Configure and run a basic Spring Boot application.

2. Apply IoC and Dependency Injection

- o Understand how Spring manages object creation and dependencies.
- o Use annotations like @Autowired, @Service, and @Repository.

3. Implement Layered Architecture

- o Structure the application using Controller, Service, and Repository layers.
- o Understand separation of concerns in backend design.

4. Develop RESTful APIs

o Create CRUD operations using Spring Boot REST controllers.

o Design APIs using proper HTTP methods (GET, POST, PUT, DELETE).

5. Use DTOs for API communication

o Implement DTO mapping to separate internal models from API responses.

6. Implement Validation

o Apply validation using Jakarta Validation annotations (@NotNull, @Email, etc.).

7. Handle Exceptions Globally

o Implement global exception handling using @ControllerAdvice.

8. Build the HealthHub Backend Module

o Develop backend services for patient/health record management.

3. Implementation/Code:

Patient.java:

```java
package com.example.healthhub.Entity;
import jakarta.persistence.*;
import lombok.*;


@Entity
@Data
@Table(name = "patients")
@AllArgsConstructor
@NoArgsConstructor
public class Patient {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private int age;
    private String gender;
    private String medicalHistory;

}
```

PatientDTO.java:

```java
package com.example.healthhub.DTO;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.*;
import lombok.Data;;

@Data
public class PatientDTO {

    private Long id;
    @NotBlank(message = "Name cannot be blank")
    private String name;
    @Min(value = 0, message = "Age cannot be negative")
    private int age;
    @NotNull(message = "Gender cannot be null")
    @NotEmpty(message = "Gender cannot be empty")
    private String gender;
    @NotNull(message = "Medical history cannot be null")
    @NotEmpty(message = "Medical history cannot be empty")
    private String medicalHistory;


}
```

PatientController.java:

```java
package com.example.healthhub.Controller;
import com.example.healthhub.Service.PatientServices;
import org.springframework.web.bind.annotation.*;
import com.example.healthhub.DTO.PatientDTO;
import com.example.healthhub.Entity.Patient;
import jakarta.validation.Valid;


@RestController
@RequestMapping("/patients")
public class PatientController {

    public final PatientServices patientServices;
    public PatientController(PatientServices patientServices) {
        this.patientServices = patientServices;
    }

    @PostMapping("/add")
    public Patient addPatient(@Valid @RequestBody PatientDTO patientDTO ) {
        return patientServices.addPatient(patientDTO);
    }

    @GetMapping("/{id}")
    public Patient getPatient(@PathVariable Long id) {
        return patientServices.getPatient(id);
    }

    @PutMapping("/update/{id}")
    public Patient updatePatient(@PathVariable Long id , @Valid @RequestBody PatientDTO patientDTO) {
        return patientServices.updatePatient(id, patientDTO);
    }

    @DeleteMapping("/delete/{id}")
    public void deletePatient(@PathVariable Long id) {
        patientServices.deletePatient(id);
    }
```

4. Learning Outcomes:

After completing this experiment, students will be able to:

1. **Understand the fundamentals of Spring Boot** and develop a basic backend application.
2. **Apply Inversion of Control (IoC) and Dependency Injection (DI)** to manage object creation and dependencies using Spring annotations.
3. **Design applications using layered architecture** (Controller, Service, Repository) to ensure clean code structure and separation of concerns.
4. **Develop RESTful APIs** by implementing CRUD operations using appropriate HTTP methods (GET, POST, PUT, DELETE).
5. **Use Data Transfer Objects (DTOs)** to separate internal data models from API responses and improve maintainability.
6. **Implement input validation** using Jakarta Validation annotations to ensure data integrity.
7. **Handle exceptions globally** using `@ControllerAdvice` to create robust and user-friendly error handling mechanisms.
8. **Build and manage backend modules** for real-world applications such as HealthHub, including patient and health record management.
9. **Develop scalable and maintainable backend services** by following best practices in Spring Boot development.